

Наследование

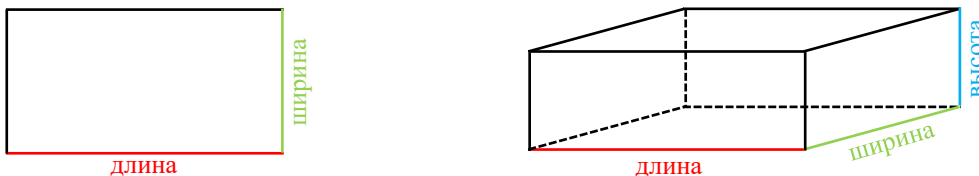
1. Основные понятия. Работа с членами класса-наследника

Наследование – один из основных принципов ООП. **Наследование позволяет определять новые классы на базе уже существующих.**

Базовый класс (класс-родитель) – класс, от которого наследуются поля и методы.

Производный класс (класс-наследник) – новый класс, определённый на базе существующего. «Наследует» поля и методы родителя. В классе-наследнике могут быть переопределены методы базового класса, а также определены свои собственные поля и методы.

В примере 1 можно видеть базовый класс Прямоугольник (Rectangle), который, как известно из математики, имеет длину и ширину, по которым можно посчитать периметр и площадь. Производным по отношению к прямоугольнику является класс прямоугольный параллелепипед (Parallelepiped), у которого также есть высота. Поскольку периметр и площадь для параллелепипеда считаются иначе, эти методы должны быть переопределены. Также для параллелепипеда как для трёхмерного объекта может быть вычислен объём.



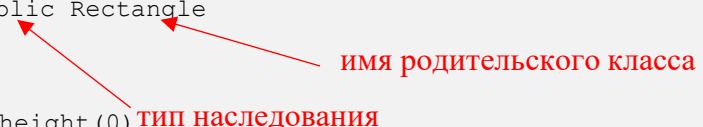
Конструктор производного класса вызывает конструктор родительского класса. При уничтожении объекта вызывается деструктор производного класса, а затем базового. Это говорит о том, что при наследовании члены базового класса не копируются в наследник, а создаются и уничтожаются средствами базового класса.

```
I'm Rectangle's constructor  
I'm Parallelepiped's constructor  
I'm Parallelepiped's destructor  
I'm Rectangle's destructor
```

Важный момент! Дочерние классы *не могут* инициализировать наследуемые поля в списке инициализации, инициализация возможна только конструктором родительского класса, от которого наследовано это поле. Значения наследуемых полей можно *изменять* в теле конструктора дочернего класса. Это не будет инициализацией, т.к. инициализация происходит ранее в конструкторе родительского класса, который вызывается конструктором дочернего класса. **При необходимости инициализации пользовательскими значениями наследованных полей вызывайте в списке инициализации конструктор родительского класса с параметрами.**

Пример 1. Базовый класс Прямоугольник и класс-наследник Параллелепипед

```
1  #include <iostream>
2
3  class Rectangle
4  {
5  protected:
6      float length;
7      float width;
8  public:
9      Rectangle() : length(0), width(0)
10     {
11         std::cout << "I'm Rectangle's constructor\n";
12     }
13     float countPerimeter()
14     {
15         return 2 * (length + width);
16     }
17     float countArea()
18     {
19         return length * width;
20     }
21     ~Rectangle()
22     {
23         std::cout << "I'm Rectangle's destructor\n";
24     }
25 };
26
27 class Parallelepiped : public Rectangle
28 {
29     float height;
30 public:
31     Parallelepiped() : height(0)
32     {
33         std::cout << "I'm Parallelepiped's constructor\n";
34     }
35     float countPerimeter()
36     {
37         return 4 * (length + width + height);
38     }
39     float countArea()
40     {
41         return 2 * (length * width + length * height + width * height);
42     }
43     float countVolume()
44     {
45         return length * width * height;
46     }
47     ~Parallelepiped()
48     {
49         std::cout << "I'm Parallelepiped's destructor\n";
50     }
51 };
52
53 int main()
54 {
55     Parallelepiped p;
56     return 0;
57 }
```



Пример 2. Инициализация наследованных полей в конструкторе с параметрами

```
1 Rectangle::Rectangle(const float &len, const float &wid) : length(len), width(wid)
2 {
3     std::cout << "I'm Rectangle's constructor\n";
4 }
5
6 Parallelepiped::Parallelepiped(const float &len, const float &wid, const float &h)
7     : Rectangle(len, wid), height(h)
8 {
9     std::cout << "I'm Parallelepiped's constructor\n";
10 }
```

Вспомним модификаторы доступа:

- `private` – члены доступны внутри класса (методам класса) и `friend`-функциям и классам;
- `protected` – члены доступны внутри класса, внутри классов-наследников и `friend`-функциям и классам;
- `public` – члены доступны как внутри, так и вне класса.

Важный момент! Если в родительском классе присутствуют члены с модификатором `private`, это не значит, что они не наследуются. Эти члены есть у класса-наследника, но *получить доступ к ним можно только через методы родительского класса*.

Поскольку объём прямоугольного параллелепипеда может быть вычислен как произведение площади основания на высоту, можно переопределить метод вычисления объёма через вызов метода родительского класса (пример 3). В данном случае обязательно нужно указать, что `countArea` относится к родительскому классу, иначе будет вызван одноимённый метод из класса-наследника. Стоит отметить, что данная реализация сработала даже в том случае, если бы длина и ширина были бы объявлены приватными, т.к. доступ к ним осуществляется через метод их класса.

Пример 3. Вычисление объёма через площадь родительского класса

```
1 float Parallelepiped::countVolume()
2 {
3     return Rectangle::countArea() * height;
4 }
```

Полный текст программы из примеров 1 и 2 с модификацией из примера 3 приведён в конце файла.

2. Модификаторы наследования

Для наследования также можно указать модификатор (спецификатор), который задаёт тип наследования. В таблице ниже приведены модификаторы доступа членов класса-наследника в зависимости от модификатора доступа члена в родительском классе и типа наследования.

		Модификатор доступа родительского класса		
		public	protected	private
Тип наследования	public	public	protected	недоступен
	protected	protected	protected	недоступен
	private	private	private	недоступен

Если вы не указываете тип наследования, то для классов по умолчанию используется **private-наследование**.

Выводы по таблице:

- Если вы планируете наследоваться без ограничений в функционале классов-наследников, при описании родительского класса **члены, к которым вы хотите ограничить доступ, определяйте как `protected`, а не `private`**, и используйте `public`-наследование.
- Если у вас нет веских причин, не используйте `protected`- и `private`- наследование, т.к. вы «закроете» доступ извне к тем членам, которые изначально были созданы доступными.

Стоит отметить, что данные правила изменения модификаторов работают только для наследуемых методов. Если переопределить метод в классе-наследнике, то к нему будет применяться тот модификатор доступа, который указан в классе-наследнике.

Зачем нужны разные типы наследования?

Существует мнение, что «`public`-наследование – это наследование интерфейса, а `private`-наследование – это наследование реализации» (интерфейс задаёт перечень методов, которые должны быть определены, но не предлагает их определения). `Protected`-наследование появилось, видимо, «чтобы было».

3. Цепное и множественное наследование

Механизм наследования позволяет делать цепочку наследования (**цепное наследование**), т.е. наследоваться от класса, который сам является наследником другого класса. Также поддерживается **множественное наследование** – наследование от нескольких классов одновременно.

Пример 4. Цепное наследование

```
1 class A
2 {
3     protected:
4         float a;
5     public:
6         A() {};
7         ~A() {};
8 };
9
10 class B : public A
11 {
12     protected:
13         float b;
14     public:
15         B() {};
16         ~B() {};
17 };
18
19 class C : public B
20 {
21     protected:
22         float c;
23     public:
24         C() {};
25         ~C() {};
26 };
```

Пример 5. Множественное наследование

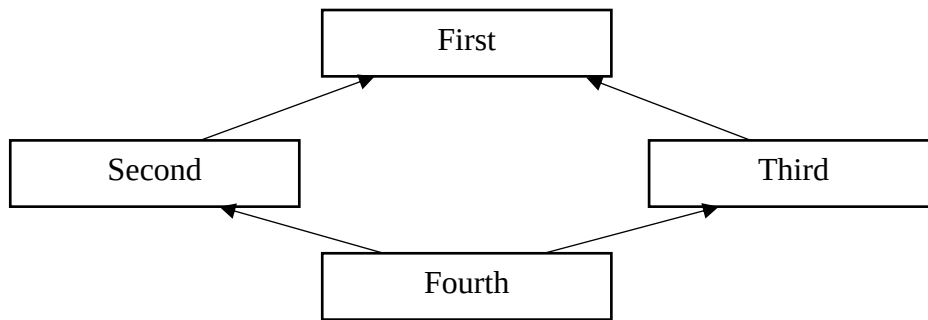
```
1 class A
2 {
3     protected:
4         float a;
5     public:
6         A() {};
7         ~A() {};
8 };
9
10 class B
11 {
12     protected:
13         float b;
14     public:
15         B() {};
16         ~B() {};
17 };
18
19 class C : public A, public B
20 {
21     protected:
22         float c;
23     public:
24         C() {};
25         ~C() {};
26 };
```

Не забывайте указывать тип наследования для каждого базового класса!

Неявные проблемы множественного наследования:

- 1) Конфликт имён, если в родительских классах есть методы с одинаковыми названиями (решается обращением через имя родительского класса как в примере 3).
- 2) Если родительские классы являются наследниками одних и тех же классов, возникает дублирование сущностей (пример 6).

В примере 6 приведено наследование классов по схеме «алмаз смерти»: класс `Fourth` является наследником классов `Second` и `Third`, которые, в свою очередь, являются наследниками класса `First`.



Пример 6. Множественное наследование с дублированием

```
1  #include <iostream>
2
3  class First
4  {
5      int a;
6  public:
7      First(const int &v = 0) : a(v)
8      {
9          std::cout << "First constructor\n";
10     }
11
12     void print()
13     {
14         std::cout << "First: a = " << a << std::endl;
15     }
16
17     ~First()
18     {
19         std::cout << "First destructor\n";
20     }
21 };
22
23 class Second : public First
24 {
25     int b;
26 public:
27     Second(const int &v1 = 0, const int &v2 = 1) : First(v1), b(v2)
28     {
29         std::cout << "Second constructor\n";
30     };
31
32     void print()
33     {
34         First::print();
35         std::cout << "Second: b = " << b << std::endl;
36     }
37
```

Продолжение примера 6.

```

38     ~Second()
39     {
40         std::cout << "Second destructor\n";
41     }
42 };
43
44
45 class Third : public First
46 {
47     int c;
48 public:
49     Third(const int &v1 = 0, const int &v2 = 1) : First(v1), c(v2)
50     {
51         std::cout << "Third constructor\n";
52     }
53
54     void print()
55     {
56         First::print();
57         std::cout << "Third: c = " << c << std::endl;
58     }
59
60     ~Third()
61     {
62         std::cout << "Third destructor\n";
63     }
64 };
65
66 class Fourth : public Second, public Third
67 {
68     int d;
69 public:
70     Fourth(const int &v1 = 0, const int &v2 = 1,
71            const int &v3 = 2, const int &v4 = 3, const int &v5 = 4)
72            : Second(v1, v2), Third(v3, v4), d(v5)
73     {
74         std::cout << "Fourth constructor\n";
75     }
76
77     void print()
78     {
79         Second::print();
80         Third::print();
81         std::cout << "Fourth: d = " << d
82                 << std::endl;
83     }
84
85     ~Fourth()
86     {
87         std::cout << "Fourth destructor\n";
88     }
89 };
90
91 int main()
92 {
93     Fourth ex1;
94     ex1.print();
95     return 0;
96 }

```

```

First constructor
Second constructor
First constructor
Third constructor
Fourth constructor
First: a = 0
Second: b = 1
First: a = 2
Third: c = 3
Fourth: d = 4
Fourth destructor
Third destructor
First destructor
Second destructor
First destructor

```

Каждый из классов имеет своё собственное поле (a – у `First`, b – у `Second`, c – у `Third`, d – у `Fourth`). У каждого из классов переопределён метод `print`, позволяющий вывести на экран значения полей каждого класса.

У класса `Fourth` мы ожидаем увидеть полный набор полей – a , b , c , d – но получаем неожиданный вариант: удвоившееся поле a , где лежат разные значения (одна копия «пришла» из класса `Second`, вторая – из `Third`).

Будьте осторожны при множественном наследовании, особенно когда наследуетесь от классов, которые написаны не вами и чью «историю» вы не можете проследить.

4. Запрет наследования

Можно запретить наследоваться от вашего класса, дописав ему спецификатор `final`. У этого спецификатора есть и другая область применения, о чём мы поговорим позднее.

Пример 7. Конечный класс

```
1 class Fourth final : public Second, public Third
2 {
3     int d;
4 public:
5     Fourth(const int &v1 = 0, const int &v2 = 1,
6           const int &v3 = 2, const int &v4 = 3, const int &v5 = 4);
7
8     void print();
9
10    ~Fourth();
11 };
```

Полнотекст из примеров 1, 2, 3.

```
#include <iostream>

class Rectangle
{
protected:
    float length;
    float width;
public:
    Rectangle() : length(0), width(0)
    {
        std::cout << "I'm Rectangle's constructor\n";
    }

    Rectangle(const float &len, const float &wid);

    float countPerimeter()
    {
        return 2 * (length + width);
    }
    float countArea()
    {
        return length * width;
    }
    ~Rectangle()
    {
        std::cout << "I'm Rectangle's destructor\n";
    }
};

class Parallelepiped : public Rectangle
{
    float height;
public:
    Parallelepiped() : height(0)
    {
        std::cout << "I'm Parallelepiped's constructor\n";
    }

    Parallelepiped::Parallelepiped(const float &len, const float &wid, const float &h);
    float countPerimeter()
    {
        return 4 * (length + width + height);
    }
    float countArea()
    {
        return 2 * (length * width + length * height + width * height);
    }
    float countVolume()
    {
        return Rectangle::countArea() * height;
    }
    ~Parallelepiped()
    {
        std::cout << "I'm Parallelepiped's destructor\n";
    }
};

Rectangle::Rectangle(const float &len, const float &wid) : length(len), width(wid)
{
    std::cout << "I'm Rectangle's constructor\n";
}
```

```
Parallelepiped::Parallelepiped(const float &len, const float &wid, const float &h)      :  
Rectangle(len, wid), height(h)  
{  
    std::cout << "I'm Parallelepiped's constructor\n";  
}  
  
int main()  
{  
    Parallelepiped p;  
    return 0;  
}
```